ΜΑΘΗΜΑ 5 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.2

Αρχεία και εξαιρέσεις

Τα δεδομένα μένουν — και τα λάθη δεν ρίχνουν το πρόγραμμα

Θεωρία σε 14 μικρά βήματα + 17 ασκήσεις στο εργαστήριο Διάρκεια: 4 ώρες (3η–6η) 25 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 2/11/2026

Τι θα ξέρεις στο τέλος

1. Να **γράφεις** κείμενο σε αρχείο και να **προσθέτεις** στο τέλος του χωρίς να σβήνεις τα παλιά.
2. Να **διαβάζεις** ένα αρχείο, ολόκληρο ή γραμμή-γραμμή.
3. Να αποθηκεύεις και να διαβάζεις πίνακες σε αρχεία **CSV** (ανοίγουν και στο Excel).
4. Να «πιάνεις» τα λάθη με **try / except**, ώστε το πρόγραμμα να **μη σταματά**.
5. Να φτιάχνεις έναν τηλεφωνικό κατάλογο που **θυμάται** τις επαφές και μετά το κλείσιμο.

Πώς δουλεύουμε

Η θεωρία είναι χωρισμένη σε **14 μικρά βήματα**. Κάθε βήμα έχει πάνω δεξιά την ετικέτα **ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** με τον αριθμό της άσκησης στο site.
Διαβάζεις τη διαφάνεια → λύνεις την άσκηση → πας στο επόμενο βήμα.

Από το εύκολο στο δύσκολο

Ασκήσεις **1–5**: εύκολες, μία εντολή τη φορά.
Ασκήσεις **6–13**: μέτριες, συνδυάζουν 2–3 βήματα.
Ασκήσεις **14–16**: δύσκολες, όλα μαζί. Η **17** είναι για το σπίτι.

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ Τα αρχεία φτιάχνονται **μέσα στον browser**, σε έναν «ψεύτικο» φάκελο. Δεν πειράζουν τον υπολογιστή σου.

Θυμάσαι από τα προηγούμενα;

ΕΚΦΩΝΗΣΗ

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι γίνεται σε κάθε περίπτωση; Πρώτα σκέψου, μετά δες τη λύση.

ΛΥΣΗ

ΚΩΔΙΚΑΣ	ΑΠΟΤΕΛΕΣΜΑ	ΓΙΑΤΙ
`int("5") + 1`	`6`	το κείμενο έγινε αριθμός
`int("πέντε")`	`ValueError`	δεν είναι αριθμός
`[1, 2, 3][5]`	`IndexError`	δεν υπάρχει θέση 5
`{"a": 1}["b"]`	`KeyError`	δεν υπάρχει κλειδί b
`10 / 0`	`ZeroDivisionError`	διαίρεση με το 0

Τα κόκκινα

Μέχρι τώρα κάθε τέτοιο λάθος **σταματούσε** το πρόγραμμα. Σήμερα θα μάθουμε να τα «πιάνουμε».

ΣΥΝΔΕΣΗ Ο τηλεφωνικός κατάλογος του Μαθήματος 4 είναι το παράδειγμά μας σήμερα.

Δύο αδυναμίες του καταλόγου μας

1 · ΚΛΕΙΝΕΙ ΤΟ ΠΡΟΓΡΑΜΜΑ, ΧΑΝΟΝΤΑΙ ΟΛΑ

```
phone = {}
phone["Άννα"] = "6971111111"
# ... κλείνουμε το πρόγραμμα ...
# το ξανανοίγουμε:
print(phone)
{}
```

2 · ΕΝΑ ΛΑΘΟΣ ΤΟΥ ΧΡΗΣΤΗ, ΤΟ ΣΤΑΜΑΤΑ

```
age = int(input("Ηλικία: "))
Ηλικία: είκοσι
ValueError: invalid literal for int()
```

Η λύση: αρχεία

Η μεταβλητή ζει στη **μνήμη** και σβήνει όταν κλείσει το πρόγραμμα. Το **αρχείο** ζει στον δίσκο και μένει.

Η λύση: try / except

Λες στην Python: «**δοκίμασε** αυτό· αν στραβώσει, κάνε **εκείνο**» — αντί να σταματήσει.

ΤΟ ΣΧΕΔΙΟ Βήματα **1–8**: αρχεία. Βήματα **9–13**: λάθη. Στο τέλος τα ενώνουμε στον κατάλογο.

Γράφω σε αρχείο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 1

```
with open("shopping.txt", "w", encoding="utf-8") as f:
    f.write("ψωμί\n")
    f.write("γάλα\n")
```

ΤΙ ΕΧΕΙ ΤΟ SHOPPING.TXT

```
ψωμί
γάλα
```

with open(όνομα, "w") as f:

Ανοίγει το αρχείο για **γράψιμο** (w = write). Το **f** είναι το αρχείο. Όταν τελειώσουν οι γραμμές με εσοχή, κλείνει **μόνο του**.

f.write(κείμενο)

Γράφει το κείμενο. Την αλλαγή γραμμής **\n** τη βάζεις εσύ, αλλιώς όλα κολλάνε σε μία γραμμή.

encoding="utf-8"

Γράφε το **πάντα**. Χωρίς αυτό, τα ελληνικά μπορεί να βγουν «κινέζικα» στα Windows.

ΠΡΟΣΟΧΗ Το **"w"** σε αρχείο που υπάρχει το **αδειάζει** πρώτα. Δεν υπάρχει αναίρεση!

Προσθέτω στο τέλος με "a"

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 2

```
with open("shopping.txt", "a", encoding="utf-8") as f:
    f.write("αυγά\n")
```

ΤΩΡΑ ΤΟ SHOPPING.TXT ΕΧΕΙ

```
ψωμί
γάλα
αυγά
```

ΤΡΟΠΟΙ ΑΝΟΙΓΜΑΤΟΣ

ΤΡΌΠΟΣ	ΤΙ ΚΆΝΕΙ	ΤΑ ΠΑΛΙΆ;
`"w"`	γράφει από την αρχή	**σβήνονται**
`"a"`	προσθέτει στο τέλος	μένουν
`"r"`	διαβάζει (βήμα 3)	μένουν

Ο κανόνας

Θέλεις να **κρατήσεις** τα παλιά; → **"a"** (append = πρόσθεσε).
Θέλεις **καθαρό** αρχείο; → **"w"**.

ΣΤΗΝ ΠΡΑΞΗ Ημερολόγια και αρχεία καταγραφής ανοίγουν πάντα με **"a"**: κάθε φορά προστίθεται κάτι στο τέλος.

Διαβάζω όλο το αρχείο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 3

```
with open("shopping.txt", encoding="utf-8") as f:
    content = f.read()

print(content)
```

```
ψωμί
γάλα
αυγά
```

Χωρίς "w" ή "a"

Αν δεν γράψεις τρόπο, το αρχείο ανοίγει για **διάβασμα**.

f.read()

Δίνει **όλο** το αρχείο σε ένα κείμενο (str), μαζί με τα \n.

Το print έξω από το with

Το content μένει και μετά το κλείσιμο του αρχείου. Το αρχείο το χρειαζόμαστε μόνο για να διαβάσουμε.

ΑΝ ΔΕΝ ΥΠΑΡΧΕΙ Αν το αρχείο δεν υπάρχει → **FileNotFoundError**. Θα δούμε πώς το πιάνουμε στο βήμα 13.

Διαβάζω γραμμή-γραμμή

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 4

```
with open("shopping.txt", encoding="utf-8") as f:
    for line in f:
        print("-", line.strip())
```

```
- ψωμί
- γάλα
- αυγά
```

ΧΩΡΙΣ STRIP: ΚΕΝΕΣ ΓΡΑΜΜΕΣ

```
with open("shopping.txt", encoding="utf-8") as f:
    for line in f:
        print("-", line)
```

```
- ψωμί

- γάλα
```

for line in f

Το for σε αρχείο δίνει **μία γραμμή κάθε φορά** — όπως το for σε λίστα δίνει ένα στοιχείο.

line.strip()

Κάθε γραμμή τελειώνει με **\n**. Το strip() το κόβει (και τα κενά στις άκρες).

Γιατί κενές γραμμές;

Η γραμμή έχει ήδη \n, και το print βάζει **ένα ακόμα**. Γι' αυτό: **πάντα strip()**.

ΜΕΤΡΗΤΗΣ Για αριθμημένες γραμμές: **n = 0** πριν τον βρόχο και **n = n + 1** μέσα, όπως στο Μάθημα 3.

Τι έχει τελικά το αρχείο;

ΕΚΦΩΝΗΣΗ

Τα τέσσερα κομμάτια τρέχουν με τη σειρά. Γράψε τι έχει το log.txt **μετά από κάθε κομμάτι**.

ΛΥΣΗ

ΚΟΜΜΆΤΙ	ΤΟ LOG.TXT ΜΕΤΆ	ΓΙΑΤΊ
`open("log.txt", "w") → write("Α\n")`	Α	νέο αρχείο
`open("log.txt", "a") → write("Β\n")`	Α, Β	πρόσθεσε
`open("log.txt", "w") → write("Γ\n")`	Γ	**τα έσβησε!**
`open("log.txt", "a") → write("Δ\n")`	Γ, Δ	πρόσθεσε

Η παγίδα

Το **τρίτο "w"** σβήνει ό,τι υπήρχε. Στο τέλος μένουν μόνο **δύο** γραμμές: Γ και Δ.

ΤΕΧΝΙΚΗ Γράψε σε στήλη τι έχει το αρχείο μετά από κάθε κομμάτι, όπως με τις λίστες.

Μια λίστα σε αρχείο, και πίσω

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 5

1 · ΓΡΑΦΩ ΤΗ ΛΙΣΤΑ

```
names = ["Άννα", "Νίκος", "Ελένη"]
with open("names.txt", "w", encoding="utf-8") as f:
    for name in names:
        f.write(name + "\n")
```

2 · ΤΗΝ ΞΑΝΑΔΙΑΒΑΖΩ

```
loaded = []
with open("names.txt", encoding="utf-8") as f:
    for line in f:
        loaded.append(line.strip())
print(loaded)
['Άννα', 'Νίκος', 'Ελένη']
```

Γράψιμο

Ένα **for** που γράφει κάθε στοιχείο σε δική του γραμμή: **name + "\n"**.

Διάβασμα

Άδεια λίστα **πριν**, και μέσα **append(line.strip())** — το μοτίβο του Μαθήματος 3.

Η ΙΔΕΑ Ό,τι ήταν στη λίστα, τώρα **μένει** στο αρχείο. Την επόμενη φορά το ξαναφορτώνεις.

Αριθμοί από αρχείο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 6

NUMBERS.TXT

```
12
7

30
```

```
total = 0
with open("numbers.txt", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line == "":
            continue        # κενή γραμμή: προσπέρασε
        total = total + int(line)
print("Άθροισμα:", total)
Άθροισμα: 49
```

Όλα είναι κείμενο

Από το αρχείο διαβάζεις **κείμενο** ("12"), όπως με το input(). Για πράξεις: **int()** ή **float()**.

Κενή γραμμή

Μετά το strip() γίνεται **""**. Το **continue** λέει «πήγαινε στην επόμενη γραμμή».

Χωρίς τον έλεγχο

int("") → ValueError. Θα το πιάσουμε και με try στο βήμα 10.

ΤΟ ΜΟΤΙΒΟ Για κάθε γραμμή: **καθάρισε** (strip) → **έλεγξε** → **μετάτρεψε** (int) → **χρησιμοποίησε**.

Αρχείο CSV: ένας πίνακας σε κείμενο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 7

```python
import csv

rows = [["Άννα", 17], ["Νίκος", 9]]

with open("grades.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "grade"])   # επικεφαλίδα
    writer.writerows(rows)               # όλες οι γραμμές
```

ΤΙ ΕΧΕΙ ΤΟ GRADES.CSV

```
name,grade
Άννα,17
Νίκος,9
```

Τι είναι το CSV

Κάθε γραμμή μια εγγραφή, οι τιμές χωρισμένες με **κόμμα**. Ανοίγει στο **Excel** σαν πίνακας.

writerow · writerows

writerow(λίστα) γράφει **μία** γραμμή. **writerows(λίστα από λίστες)** γράφει **όλες**.

newline=""

Γράφε το πάντα στα CSV. Χωρίς αυτό, στα Windows μπαίνει μια κενή γραμμή ανάμεσα σε κάθε εγγραφή.

ΕΧΕΙ Αν το Excel δείχνει «κινέζικα», γράψε **encoding="utf-8-sig"**.

Διαβάζω αρχείο CSV

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 8

```
import csv

with open("grades.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader)              # προσπερνώ την επικεφαλίδα
    for row in reader:
        print(row[0], "→", row[1])
Άννα → 17
Νίκος → 9
```

ΚΑΘΕ ROW ΕΙΝΑΙ ΜΙΑ ΛΙΣΤΑ

ΓΡΑΜΜΉ ΑΡΧΕΊΟΥ	ROW	ROW[1]
Άννα,17	['Άννα', '17']	'17'
Νίκος,9	['Νίκος', '9']	'9'

next(reader)

Η πρώτη γραμμή είναι **επικεφαλίδες**, όχι δεδομένα. Το next() την «τραβάει» για να ξεκινήσει ο βρόχος από τη 2η.

Πάλι κείμενο

Το **row[1]** είναι "17", όχι 17. Για πράξεις: **float(row[1])**.

ΘΥΜΗΣΟΥ row[0] = 1η στήλη, row[1] = 2η στήλη — θέσεις από το 0, όπως σε κάθε λίστα.

Διαβάζω ένα μήνυμα λάθους

```
Traceback (most recent call last):
  File "age.py", line 2, in <module>
    age = int(input("Ηλικία: "))
ValueError: invalid literal for int() with base 10: 'είκοσι'
```

Από κάτω προς τα πάνω

Η **τελευταία** γραμμή λέει **τι** έγινε (το όνομα του λάθους). Το **line 2** λέει **σε ποια γραμμή**.

Εξαίρεση

Έτσι λέγεται ένα λάθος που γίνεται **ενώ τρέχει** το πρόγραμμα. Το όνομά του (ValueError κ.λπ.) θα το χρειαστούμε στο επόμενο βήμα.

ΤΑ ΠΙΟ ΣΥΧΝΑ

ΛΆΘΟΣ	ΠΌΤΕ ΓΊΝΕΤΑΙ
`ValueError`	int("abc"), float("")
`ZeroDivisionError`	διαίρεση με το 0
`FileNotFoundError`	το αρχείο δεν υπάρχει
`IndexError`	θέση λίστας που δεν υπάρχει
`KeyError`	κλειδί λεξικού που δεν υπάρχει

ΣΥΜΒΟΥΛΗ Μην τρομάζεις με το Traceback: διάβασε μόνο την **τελευταία γραμμή** και το **line N**.

Δοκίμασε — κι αν στραβώσει...

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 9

```
try:
    age = int(input("Ηλικία: "))
    print("Του χρόνου:", age + 1)
except ValueError:
    print("Δώσε αριθμό, όχι γράμματα!")

print("Τέλος")
Ηλικία: είκοσι
Δώσε αριθμό, όχι γράμματα!
Τέλος
```

Πώς δουλεύει

Τρέχουν οι γραμμές του **try**. Αν μία βγάλει ValueError, η Python **πηδάει** αμέσως στο **except**. Μετά το πρόγραμμα **συνεχίζει** κανονικά.

Αν δεν γίνει λάθος

Τρέχει όλο το try και το except **προσπερνιέται**.

Γράφε το όνομα

except ValueError: και όχι σκέτο except:. Έτσι πιάνεις μόνο το λάθος που περιμένεις.

ΕΣΟΧΕΣ Όπως στο if: μετά το **try:** και το **except ...:** οι γραμμές μπαίνουν με **4 κενά** μέσα.

Ένα except για κάθε λάθος

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 10

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print("Αποτέλεσμα:", a / b)
except ValueError:
    print("Μόνο αριθμοί!")
except ZeroDivisionError:
    print("Διαίρεση με το 0!")
```

ΤΙ ΕΜΦΑΝΙΖΕΙ

ΕΊΣΟΔΟΙ	ΕΜΦΑΝΊΖΕΙ
10 και 4	Αποτέλεσμα: 2.5
10 και x	Μόνο αριθμοί!
10 και 0	Διαίρεση με το 0!

Ποιο except τρέχει;

Η Python κοιτάει τα except **με τη σειρά** και τρέχει **μόνο** αυτό που ταιριάζει με το λάθος.

ΘΑ ΤΟ ΔΕΙΣ ΚΑΙ ΕΤΣΙ Μερικές φορές ακολουθεί **finally:** — γραμμές που τρέχουν **πάντα**, με λάθος ή χωρίς.

Ρώτα ξανά μέχρι να είναι σωστό

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 11

```
while True:
    try:
        n = int(input("Ακέραιος: "))
        break
    except ValueError:
        print("Λάθος, ξαναδοκίμασε.")

print("Έδωσες", n)
Ακέραιος: πέντε
Λάθος, ξαναδοκίμασε.
Ακέραιος: 5.5
Λάθος, ξαναδοκίμασε.
Ακέραιος: 5
Έδωσες 5
```

while True

Ο βρόχος που τρέχει μέχρι να του πεις **break** (Μάθημα 2).

break μέσα στο try

Φτάνει εκεί **μόνο αν πέτυχε** το int(). Τότε βγαίνουμε από τον βρόχο.

except

Αν απέτυχε, τυπώνει μήνυμα και ο βρόχος **ξαναρωτά**.

Κράτα το!

Είναι το πιο χρήσιμο κομμάτι κώδικα της ημέρας: κάθε φορά που ζητάς αριθμό.

ΓΙΑΤΙ ΤΟ 5.5; Απορρίφθηκε γιατί ζητάμε **int**. Για δεκαδικούς, γράψε **float(input(...))**.

Όταν το αρχείο δεν υπάρχει

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 12

```
try:
    with open("notes.txt", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("Δεν υπάρχει ακόμα αρχείο.")
```

ΤΟ ΜΟΤΙΒΟ ΤΟΥ ΚΑΤΑΛΟΓΟΥ

```
phone = {}
try:
    with open("phone.csv", newline="", encoding="utf-8") as f:
        for row in csv.reader(f):
            phone[row[0]] = row[1]
except FileNotFoundError:
    pass          # 1η φορά: ξεκινάμε με κενό κατάλογο
```

Η πρώτη φορά

Την πρώτη φορά που τρέχει ένα πρόγραμμα, το αρχείο του **δεν έχει φτιαχτεί** ακόμα. Δεν είναι λάθος, είναι αναμενόμενο.

pass

Σημαίνει «**μην κάνεις τίποτα**». Το λεξικό μένει άδειο και το πρόγραμμα συνεχίζει.

Το σχέδιο

Φόρτωσε στην αρχή (αν υπάρχει) → **δούλεψε** → **αποθήκευσε** στο τέλος.

ΣΥΜΒΟΥΛΗ Αν το πρόγραμμα «δεν βρίσκει» ένα αρχείο που βλέπεις, τρέχει από άλλον φάκελο. Στο VS Code: **File** → **Open Folder**.

Ποια γράμματα θα εμφανιστούν;

```
try:
    x = int(input())
    print("Α")
    y = 10 / x
    print("Β")
except ValueError:
    print("Γ")
except ZeroDivisionError:
    print("Δ")
print("Ε")
```

ΕΚΦΩΝΗΣΗ

Για κάθε είσοδο, γράψε με τη σειρά τα γράμματα που εμφανίζονται.

ΛΥΣΗ

ΕΊΣΟΔΟΣ	ΕΜΦΑΝΊΖΕΙ	ΓΙΑΤΊ
5	Α Β Ε	κανένα λάθος
abc	Γ Ε	λάθος στο int()
0	Α Δ Ε	λάθος στη διαίρεση

Η ουσία

Μόλις γίνει λάθος, το try **σταματά εκεί**: ό,τι ακολουθεί μέσα του δεν τρέχει. Το **Ε** είναι έξω από το try, οπότε τρέχει **πάντα**.

ΔΟΚΙΜΑΣΕ Τρέξε τον κώδικα στο «🧪 Ελεύθερο» του εργαστηρίου με τις τρεις εισόδους.

Αποθηκεύω ολόκληρο λεξικό

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 13

```
import pickle

phone = {"Άννα": "6971111111", "Νίκος": "6972222222"}

with open("phone.dat", "wb") as f:
    pickle.dump(phone, f)         # αποθήκευση

with open("phone.dat", "rb") as f:
    loaded = pickle.load(f)       # φόρτωση

print(loaded["Νίκος"])
6972222222
```

Δύο εντολές

pickle.dump(δομή, f) αποθηκεύει, **pickle.load(f)** φορτώνει. Το λεξικό γυρίζει **όπως ήταν**.

"wb" και "rb"

Το b σημαίνει **δυαδικό** αρχείο: δεν είναι κείμενο, δεν ανοίγει στο Σημειωματάριο. Γι' αυτό **χωρίς** encoding.

CSV ή pickle;

CSV όταν θες να το ανοίξεις στο Excel. **pickle** όταν θες απλώς να «παγώσεις» μια δομή Python.

ΑΣΦΑΛΕΙΑ Ποτέ **pickle.load** σε αρχείο που σου έστειλε άγνωστος.

Οι 17 ασκήσεις, βήμα-βήμα

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
1	Το πρώτο μου αρχείο	Εύκολο	5
2	Πρόσθεσε στο τέλος	Εύκολο	6
3	Διάβασε το μήνυμα	Εύκολο	7
4	Γραμμή-γραμμή	Εύκολο	8
5	Σώσε τη λίστα	Εύκολο	10
6	Άθροισμα από αρχείο	Μέτριο	11
7	Γράψε ένα CSV	Μέτριο	12
8	Διάβασε ένα CSV	Μέτριο	13
9	Ηλικία χωρίς σφάλμα	Μέτριο	15
10	Διαίρεση χωρίς σφάλμα	Μέτριο	16
11	Ρώτα ξανά	Μέτριο	17
12	Φόρτωσε αν υπάρχει	Μέτριο	18
13	pickle: πάγωσε το λεξικό	Μέτριο	20
14	Ασφαλής είσοδος βαθμών	Δύσκολο	17, 22
15	Στατιστικά από CSV	Δύσκολο	13, 23
16	Κατάλογος που θυμάται	Ο στόχος	18, 24
17	Ημερολόγιο	Για το σπίτι	6, 8

ΣΤΟ SITE Εργαστήριο → **Μάθημα 5**. Κάθε άσκηση έχει το «📖 Στις διαφάνειες» που ανοίγει κατευθείαν τη σωστή διαφάνεια.

Ασφαλής είσοδος βαθμών

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 14

ΕΚΦΩΝΗΣΗ

Διάβασε **5 βαθμούς** (0–20). Αν ο χρήστης γράψει κάτι που δεν είναι αριθμός ή είναι **εκτός ορίων**, εμφάνισε «Λάθος» και ξαναρώτα **τον ίδιο** βαθμό. Στο τέλος εμφάνισε τον μέσο όρο.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
grades = []
while len(grades) < 5:
    try:
        g = float(input(f"Βαθμός {len(grades) + 1}: "))
        if g < 0 or g > 20:
            print("Λάθος: εκτός ορίων 0-20")
        else:
            grades.append(g)
    except ValueError:
        print("Λάθος: δώσε αριθμό")

avg = sum(grades) / len(grades)
print(f"Μέσος όρος: {avg:.2f}")
```

```
Βαθμός 1: 15
Βαθμός 2: abc
Λάθος: δώσε αριθμό
Βαθμός 2: 25
Λάθος: εκτός ορίων 0-20
Βαθμός 2: 12
...
Μέσος όρος: 14.90
```

Γιατί while len(grades) < 5

Μετράμε τους **σωστούς** βαθμούς, όχι τις προσπάθειες. Με for range(5) θα χάναμε βαθμό σε κάθε λάθος.

ΑΠΟ ΠΟΙΑ ΒΗΜΑΤΑ **10** try / except · **12** ρώτα ξανά · Μάθημα 3: **append**, **sum / len**.

Στατιστικά από αρχείο CSV

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 15

ΕΚΦΩΝΗΣΗ

Διάβασε το **grades.csv** και εμφάνισε πλήθος μαθητών, **μέσο όρο** και **ποιοι πέρασαν** (≥ 10). Αν το αρχείο λείπει, εμφάνισε μήνυμα αντί για σφάλμα.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
import csv

names = []
grades = []
try:
    with open("grades.csv", newline="", encoding="utf-8") as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            names.append(row[0])
            grades.append(float(row[1]))
except FileNotFoundError:
    print("Δεν βρέθηκε το grades.csv")

if len(grades) > 0:
    print("Μαθητές:", len(grades))
    print(f"Μέσος όρος: {sum(grades) / len(grades):.2f}")
    passed = []
    for i in range(len(names)):
        if grades[i] >= 10:
            passed.append(names[i])
    print("Πέρασαν:", ", ".join(passed))
```

```
Μαθητές: 5
Μέσος όρος: 12.60
Πέρασαν: Άννα, Μαρία, Γιώργος
```

", ".join(λίστα)

Ενώνει τα ονόματα με **", "** ανάμεσα: ['Άννα', 'Μαρία'] → Άννα, Μαρία

Από ποια βήματα;

8 csv.reader · **13** αρχείο που λείπει · Μάθημα 3: **δύο λίστες**

ΔΟΚΙΜΑΣΕ Στο εργαστήριο ο έλεγχος τρέχει και **χωρίς** το αρχείο: πρέπει να δεις το μήνυμα, όχι Traceback.

Κατάλογος που θυμάται

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 16

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```python
import csv
FILE = "phone.csv"

phone = {}
try:                                        # 1 · φόρτωση
    with open(FILE, newline="", encoding="utf-8") as f:
        for row in csv.reader(f):
            phone[row[0]] = row[1]
except FileNotFoundError:
    pass

while True:                                 # 2 · μενού
    choice = input("\n1.Προσθήκη 2.Αναζήτηση 3.Όλοι 0.Έξοδος: ")
    if choice == "1":
        name = input("Όνομα: ")
        phone[name] = input("Τηλέφωνο: ")
    elif choice == "2":
        name = input("Όνομα: ")
        print(phone.get(name, "Δεν βρέθηκε"))
    elif choice == "3":
        for name in sorted(phone):
            print(f"{name}: {phone[name]}")
    elif choice == "0":                     # 3 · αποθήκευση
        with open(FILE, "w", newline="", encoding="utf-8") as f:
            csv.writer(f).writerows(phone.items())
        print("Αποθηκεύτηκαν", len(phone), "επαφές.")
        break
    else:
        print("Λάθος επιλογή")
```

1 · Φόρτωση

Αν υπάρχει το phone.csv, γεμίζει το λεξικό. Αν όχι, **pass** (βήμα 13).

2 · Μενού

Ίδιο με το Μάθημα 4.

3 · Αποθήκευση

Το **phone.items()** δίνει ζευγάρια (όνομα, τηλέφωνο) → **writerows** τα γράφει όλα (βήμα 7).

ΔΟΚΙΜΑΣΕ Πρόσθεσε 2 επαφές, βγες με 0, ξανατρέξε και πάτα 3. Οι επαφές είναι εκεί!

Τι κρατάμε

ΘΈΛΩ ΝΑ…	ΓΡΆΦΩ
γράψω (σβήνει τα παλιά)	`with open(όνομα, "w", encoding="utf-8") as f:`
προσθέσω στο τέλος	`with open(όνομα, "a", encoding="utf-8") as f:`
γράψω μια γραμμή	`f.write(κείμενο + "\n")`
διαβάσω όλο	`f.read()`
διαβάσω γραμμή-γραμμή	`for line in f: … line.strip()`
γράψω / διαβάσω CSV	`csv.writer(f) · csv.reader(f)`
πιάσω ένα λάθος	`try: … except ValueError: …`
ρωτήσω ξανά	`while True: try: … break`
χειριστώ αρχείο που λείπει	`except FileNotFoundError:`

Θυμάται και αντέχει

Με τα **αρχεία** το πρόγραμμα **θυμάται**. Με το **try** **αντέχει** τα λάθη του χρήστη.

Επόμενη φορά · Δευ 9/11

Συναρτήσεις: το «ρώτα ξανά μέχρι να είναι σωστό» γίνεται μία εντολή που καλείς όσες φορές θες.

ΓΙΑ ΤΟ ΣΠΙΤΙ Άσκηση **17** στο εργαστήριο: ημερολόγιο με "a" και αριθμημένες γραμμές (βήματα 2 και 4).